

AED - Atividade: Análise de complexidade

Professor: Thiago Goveia	Alunos (dupla):	Valor: 0,4 pto
------------------------------------	------------------------	--------------------------

1. Verdadeiro ou Falso (0,1pt):

- a) () A expressão “pior caso” se refere ao maior tamanho N de uma entrada, que pode ser executado por um algoritmo;
- b) () A notação Ômega (Ω) define o limite assintótico inferior de uma função (ou algoritmo);
- c) () O custo de execução de um algoritmo linear, isto é, $O(n)$ não depende do tamanho da entrada, ou seja, o custo é constante;
- d) () A busca binária permite encontrar um item em um array qualquer em tempo logarítmico ($O(\log n)$);
- e) () O custo de inserção em um array ordenado geralmente é menor que em um array desordenado;
- f) () No limite assintótico Θ são avaliadas 3 constantes (c_1 , c_2 e m) para limitar a função analisada. Na notação $O(n)$ são avaliadas apenas duas constantes (c e m);
- g) () O custo de uma função pode ser dado em termos do tempo de execução (quantidade de operações) ou do espaço de armazenamento utilizado;
- h) () O caso médio raramente é utilizado pois depende de uma análise de distribuição de probabilidades, o que é algo complexo de se fazer;

2. Considere os métodos `bubbleSort` e `selectionSort` a seguir. Os trechos em negrito se referem às comparações e os trechos sublinhados às movimentações (ou swaps) (0,2pt).

```
public void bubbleSort(int[] v){
    int n = v.length;
    for (int i = 0; i < n-1; i++){
        for (int j = n-1; j > 0; --j){
            if (v[j] < v[j-1]){
                swap(v, j, j-1);
            }
        }
    }
}
```

```
public void selectionSort ( int[] v ) {
    int menor, index_menor;
    int n = v.length;
    for(int k = 0; k < n-1; k++){
        menor = v[k];
        index_menor = k;
        for (int i = k+1; i < n; i++) {
            if(v[i] < menor){
                menor = v[i];
                index_menor = i;
            }
        }
        swap(v, k, index_menor);
    }
}
```

Defina as funções de complexidade $C(n)$ e $M(n)$ para cada um, sabendo que $C(n)$ é o número de comparações e $M(n)$ é o número de movimentações, isto é, o número de vezes que a função swap é executada.

Dica: Vimos que nos algoritmos bubbleSort e selectionSort o número de comparações diminui uma unidade a cada iteração, começando por exemplo, com 5, e progredindo para 4, 3, 2, 1 (Ver tabelas de ordenação). Para se obter então a função $C(n)$ é necessário recorrer à fórmula da soma de progressão aritmética. Se o número de comparações vai de n até 1, a fórmula é:

$$\sum_{i=1}^n i = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

3. Marque alternativa FALSA (0,1pto):

- A. Um algoritmo de custo linear (Ex: $100n$) pode levar mais tempo para executar que um algoritmo quadrático (Ex: $2n^2$);
- B. Um algoritmo de complexidade $O(n!)$ é dito de complexidade exponencial, apesar de $O(n!)$ ter comportamento muito pior do que $O(2^n)$;
- C. É possível resolver um determinado problema X com algoritmos de diferentes complexidades;
- D. Se o custo de um programa F é $f(n)$ e o de um programa G é $g(n) = 3f(n)$, tem-se que o comportamento assintótico de $g(n)$ é pior que o de $f(n)$, isto é, $O(f(n)) < O(g(n))$;
- E. A complexidade logarítmica é obtida em algoritmos que geralmente usam uma estratégia de divisão e conquista;